



P2P CNY settlement — WeChat Pay / Alipay demo (step by step)

P2P settlement · WeChat Pay / Alipay (CNY rails)

Step-by-step demo guide

captured 2026-10-09T07:39:57.135Z · swap `swap-91748e19f7da5216`

Status: **built and captured**. Every screenshot below was taken from the real wallet UI driving the real `p2p-engine` on 2026-10-09 by `e2e/capture-p2p-cny.mjs` — two fresh wallets, one engine, no mocks. Companion documents: [docs/p2pcny.md](#) (design & threat model), [docs/p2p.md](#) (protocol), [docs/p2p-demo/p2p.md](#) (PayPal demo guide), engine source `services/p2p-engine/src/`.

0. What this demo proves (read first)

- 1. **A CNY swap needs no API and no webhook.** The buyer transfers money to the seller's own WeChat/Alipay/bank account; the *seller's confirmation* is the fiat signal. The engine never touches CNY — it enforces `instructions → proof → confirm` and releases tokens only after payment was confirmed (by the seller, or by the admin arbiter after a dispute).
- 2. **The buyer cannot fake the money.** `PAYMENT_CLAIMED` is only the buyer's claim (流水号 + optional receipt image). Tokens stay locked until the seller signs `confirm` at `PAYMENT_CLAIMED` — the UI repeats this as a red warning above the *Confirm received* button.
- 3. **PII never reaches the order book or the chain.** `GET /public/orders` carries only the trade terms. The seller's 收款 account is revealed party-scoped by `POST /swaps/:id/payment-instructions`, and only a per-swap `remark` code plus (optionally) a receipt `sha256` are stored on the swap event.
- 4. **Rails are a data field, not a fork.** ``wantRail ∈ {paypal, wechat, alipay, bank}`` decides the whole fiat leg: PayPal keeps `payment_send / payout` (invoice + payout webhooks), the CNY rails use `instructions / proof / confirm / complete ( state.ts:12-24 )`.
- 5. **Every mutating call is signed with the party's PQ (ML-DSA-87) `did:key`** — the same wallet key that signs chain transactions — over the sha256 of the canonical (sorted top-level keys) JSON body, plus a nonce + timestamp (±5 min): replay-protected, rate-limited to 10 signed requests per DID per minute (`sign.ts:17-19`). Classic Ed25519/secp256k1 dids still verify for compatibility, but new keys are PQ. The demo's final two steps — `release` and `complete` — are signed by the engine signer (`SETTLEMENT_ENGINE_DID`, an ML-DSA did:key ≈3.5 kB); nothing fires them automatically yet, so the capture script stands in for that operator job (\$6.1).

1. State machine — who advances each step

#	Status	Action (HTTP)	Who must sign	CNY-specific evidence
---	--------	---------------	---------------	-----------------------

1	MATCHED	POST /orders/:id/match	buyer	receiveAddress (token payout address)
2	ESCROW_LOCKED	POST /swaps/:id/transitions action=escrow_lock	seller	txHash (escrow lock proof)
3	PAYMENT_PENDING	POST /swaps/:id/payment-instructions	buyer	remark (12 hex chars, per-swap)
4	PAYMENT_CLAIMED	POST /swaps/:id/proof	buyer	txId 流水号, optional receipt + receiptSha256
5	PAYMENT_VERIFIED	POST /swaps/:id/confirm	seller	the seller's own statement check
6	ESCROW_RELEASED	POST /swaps/:id/transitions action=release	engine	txHash (release proof)
7	COMPLETED	POST /swaps/:id/transitions action=complete	engine	— (seller already holds the CNY)

Freeze paths: either party may POST /swaps/:id/dispute while PAYMENT_PENDING / PAYMENT_CLAIMED ; the admin then resolves with outcome=release|refund (server.ts:765-809). expire / refund / cancel stay available before release so a failed deal can always return the tokens.

2. The rails and how they are configured

Rail (wantRail)	Buyer pays	Seller verifies	Tokens released by
paypal	PayPal invoice	PayPal webhook → verify	engine payout → COMPLETED
wechat	WeChat Pay to seller's WeChat ID	seller's own 流水 + remark	engine release + complete
alipay	Alipay to seller's account	seller's own 流水 + remark	engine release + complete
bank	bank transfer + remark	bank statement + remark	engine release + complete

Engine environment (services/p2p-engine/src/server.ts):

```
SETTLEMENT_CNY_RAILS=wechat,alipay,bank # unset → all three; "" → rail off
SETTLEMENT_CNY_REMARK_TTL=900           # seconds → payBy deadline in instructions
SETTLEMENT_ENGINE_DID=did:key:z446Gmp1... # ML-DSA-87 did:key (≈3.5 kB); only signer of release/complete/verify
SETTLEMENT_ENGINE_PUBKEY=050101010a20... # prefixed ML-DSA pub (hex) → 2-of-3 escrow address derivation
SETTLEMENT_STORE=mem|pg                 # mem for demos, pg for production
SETTLEMENT_ADMIN_TOKEN=adm              # /orders, /swaps, dispute/resolve
CORS_ORIGIN=http://localhost:18081      # wallet web app origin
```

3. Running the demo yourself

```
# 1. workspace packages the engine imports
for pkg in did p2p-protocol record-sig; do ( cd packages/$pkg && npm run build ); done

# 2. engine bundle (esbuild → dist/server.bundle.mjs)
( cd services/p2p-engine && npm run build && npm run bundle )

# 3. wallet web app with the engine URL inlined by `expo export`
( cd expo-app && EXPO_PUBLIC_P2P_ENGINE_URL=http://localhost:18089 npm run web:build )
echo http://localhost:18089 > e2e/web-build/.p2p-url

# 4. capture: starts engine + web server, drives two fresh wallets end to end
node e2e/capture-p2p-cny.mjs          # KEEP=1 ... leaves the servers running
```

The capture writes 14 numbered PNGs to

`docs/p2p-demo/assets/screenshots/` and a machine-readable run report to

`e2e/demo-output/p2p-cny-capture.json`. It is self-contained: no L0/L1, no

PayPal — a CNY swap has no chain-side dependency beyond the escrow lock proof

the engine skips when `deps.chain` is unset.

Run facts used in the screenshots below:

```
{ "swapId": "swap-91748e19f7da5216", "txId": "420322BE2434E177",
  "remark": "910fc8a8926f", "capturedAt": "2026-10-09T07:39:57Z" }
```

4. Walkthrough

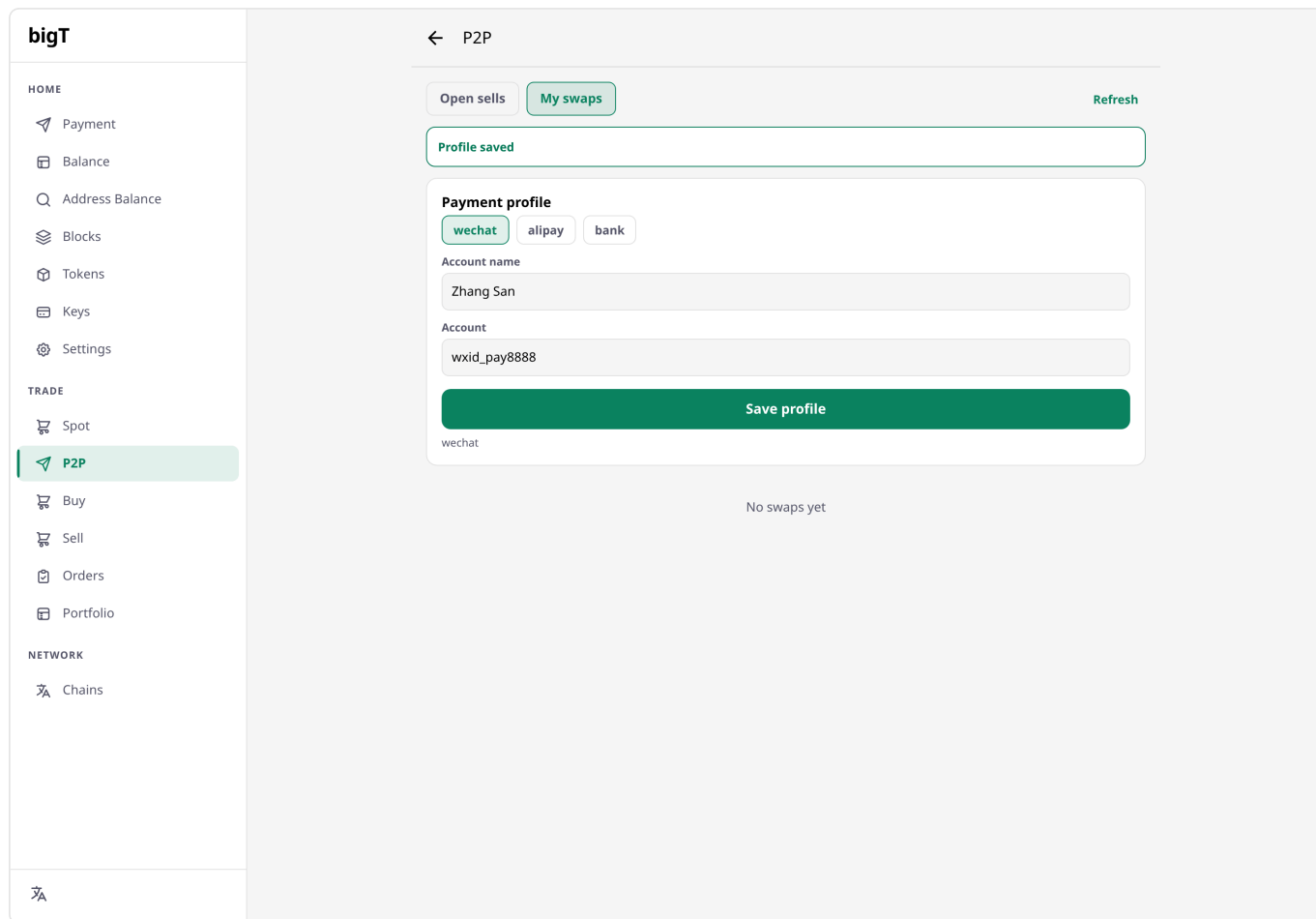
Step 1 — Seller saves a collection profile

The seller opens **P2P → My swaps**, picks the rail (`wechat`), and enters the

收款 name and account (WeChat ID / Alipay account / card number). The profile is

stored server-side under the seller's DID and is **not** part of the public

order book — it is only revealed to the matched buyer by the instructions call.



Step 2 — Seller lists 10 USDT for 715 CNY on the wechat rail

Open sells → **List a sell order**: token `USDT` , amount `10` , price `715` ,
currency `CNY` , chain `L0` , payment method **wechat** (selecting a CNY rail
switches the currency to CNY; `paypal` remains available for USD deals). The
signed `POST /orders` payload carries no PII:

```
{ "sellerDid": "did:key:z446G...(ML-DSA did, ≈3.5 kB)", "giveToken": "USDT",  
  "giveAmount": "10", "giveChain": "L0", "wantCurrency": "CNY",  
  "wantAmount": "715", "wantRail": "wechat", "validUntil": 1791510000,  
  "nonce": "...16+ chars...", "timestamp": 1791509100000,  
  "signature": "...hex SignatureBundle over sha256(canonical JSON)..." }
```

bigT

HOME

Payment

Balance

Address Balance

Blocks

Tokens

Keys

Settings

TRADE

Spot

P2P

Buy

Sell

Orders

Portfolio

NETWORK

Chains

← P2P

Open sells

My swaps

Refresh

List a sell order

Token

USDT

Amount

10

Price

715

Currency

CNY

Chain

L0

Payment method

paypal

wechat

alipay

bank

List order

No open sell orders

Step 3 — Buyer matches the order

The buyer (fresh wallet, no profile needed) sees the order in **Open sells**

and opens the buy panel. On a CNY order the PayPal-only fields are **absent** —

`paypalAccount` / `buyerEmail` are not required (`server.ts:294`), only a receive address is:

bigT

HOME

Payment

Balance

Address Balance

Blocks

Tokens

Keys

Settings

TRADE

Spot

P2P

Buy

Sell

Orders

Portfolio

NETWORK

Chains

0.00

Currency

USD

Chain

L0

Payment method

paypal

wechat

alipay

bank

List order

10 USDT

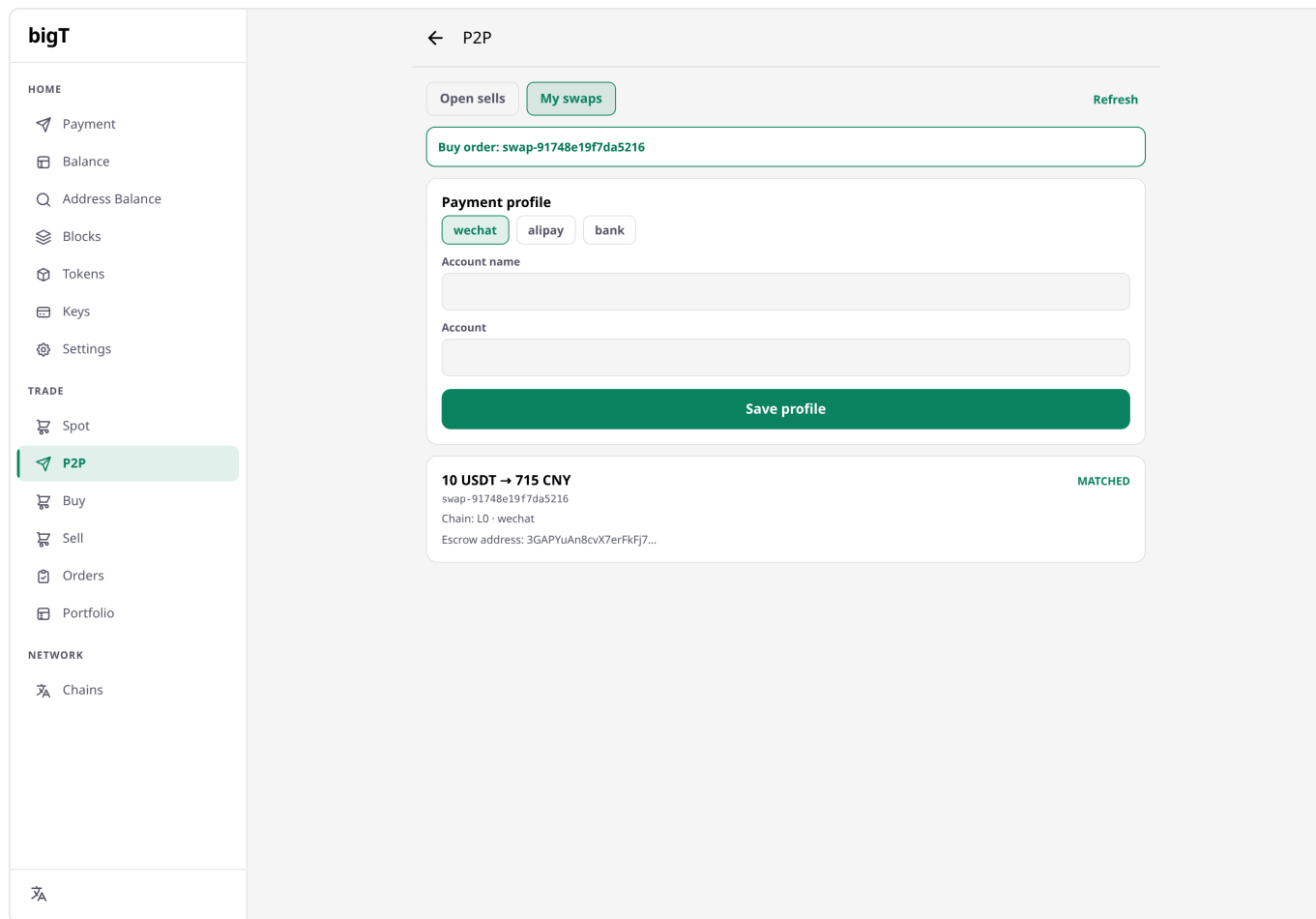
Chain: L0 - wechat

d1d:key:z446Gmp1fqlFYjzUaoZ57kdTHfvByq515CDmq444We5jZqmRU85PdF3TnGe3VqHV5B3NHCBgG1xswJ20tAmNDnEH5ScRBV1xt3mtK2PrGUVc85xuvJ3gu7orgzPH715g518jUFvqpASB2o5fsg7vTXVefkYaov3PpQJ34V4suppgAnZFLC2N6dHnNyWdnnvXvHTL81xw9APKjJH8tao1T13Hn6asG5dsuV9XLFHMEJN3BdemkDevo5XRCYbd3u4zd81UUEcMgZugZjY7KDZ1Y5aj9j4JL3Mep11CYkj rKDxoo2defbH5wJNzXWY75a7Tc1vn3JQz51hRzcVYCK12FecK3JMcK24erbf4nzdVY6BgMFzbYFZ677n31PpD174Jvd057nPY5ga0L8EYjaJbb1tjmHzJb179xhJZnZ3sAPhVfCrtpmG29ZGZd1fZXKfwmkCRZ2TW55le5YQctjH8W1Bdres6QEU7RHTcV0Zdctg4e0yosKUA9mZYSa9qVfMnJ51t4Sye3U86wXN17cyKkYXR0YyPPH5wRbeXF9o5EV89UmuA92xyGGA3g6j4mGzgmMuzU5rQWqQTChdx12yzP5yz4YawaHmq54rHvyWdtQjCKbtS1MBGf7ZLT2cZnVaMc8ymNMXc4LSMjc3cq8wGbYpCFqdh6Kxx9MA38xZjm8WjBmHAfHYo5X2bph1R9ez18FJnc9BR173XQnczysggAovYcDCTK5SmZyg9MDVAwHVK6Z0ugwhTmWkt5jvw7NTPmAMfssvZQ28VJNn6sCARUFaWBgwrgQTGTZKsLMPADXg6LAhrRyZtMkcej1xtfG9RpRZvW3aPVZp3k7tFXqCNSv59szLxLlPainaR616vftAsPxwZKao33Psg6EBzrvfHMH4xpFDGUFFU9cxcnH1BEzneftboYcckaRu0nv7BMHzjXCuPYBgqBGbselgelKtffkKReF1Nv5S5ffVC4BRnA4BNWboJLs2zJ75bkQZw5QqLRzQZfo3pp2vgc5nCLdkqYSLNNqJW14RFchpAUuxLu27jBhG4kew1L1NntRaz11755eFGT6G3G5NKRopnBsc4uRA6YtHaytM23KnlPolwsY1BvnpqMMLzF4Wj0bzZHNHs7jCT8WAAr3odpdR12RBSNgZox8BxzF2941qBX4jRPPaZ1uWYh5Fo8504qmG0jAKS9j87ky8ZYHFMIEmj6ntFXABZ2emsQ133unyUFN4573xZ73W4QD5TACpdP1hrkvZFLLeJh95eMhByYRGtcc2UNwwMnJ1vRkMRHChJwck86DppaNPAP69hxrDFGPEPZHs2UDRZcSA8uWd19MRudnTJ2AqWshKZbj7HKKpT3cU0BPCLFLJA9mZ58mjjeur5LDtmeXgFunXB7MQVXFNDjHuhw5Qp2hnc290jxpTsmLQWmqWQcYw0jnbCpoAe1TzwQE311nWjFxlFDES8P16fh0ryKFepFF1TQjAdTcmEaFaFU18Yb59FfiP5K7ZUQW9aURc3wFgYG5T5Gjhx47HhmcHFBFL9uSCAU4Z8ug5LgV18kZcrt3C8moJ2Hlv156R9ERXce4mFH0gq4RNPdxroqjnwffM1Em1D6LPT8b8Fkf5FTU5cxoa4DofwF2G6AcxTGUmkinnn9grZ5geLoJ1DnPQz2jjpK1428LxPRfetsRp1MpfmJdlbheXGyMTLQ7aJxz6dmJ4fKG8RjZgwW55kp7et1mwM8SYJkXmH3a3UXWJGHSa0vtLHCM6k4KXWtLeS1Y978AJCjx84xkcdPegunfataCSPU84JKVGZ34dNGJpQKApMvL1YExy5nZ1voMZXvG02bkyPKLfbeiXezdrw83SxwactJ1GRdZ7H6ZEPhoXsYVZuqUBQ8XaYU17ZxydKQ6HE1DQ41ABuXHGZhnBv54nAwk1qebqLZTRHcGSy2MotT1aaj1YKZ508j1g1Vvntj1v1cFhm7cxtaTeq9CRm9WltqPwX7daBqB8Zhw1Qq5XkoKDVm3nXKsok69E1QA3yDHJEVnapCA6e3cX9vhWq3FwV4TMCimZn5Dsl1v9y9QuysyqZCJMK8coz63Hhnci6fTcBKfCK6FBx5Sh1owr1NeZMp8zn1mFcl2xEqMqapuvXqT0oZrZeAbAH0D5KhmbY8LKZEGxsfgfTqe6CFtFJ1TV6mwEZZX9bfPbKVMXLj8Rpja7EYNgbbHmZVnawXQnancF7Y1lCt97ZLsm35NwfbnnfGdRZVrfG5CuFCGN8Xr52fjLQH24yE6tzzXvgoXBE6tkaJ8eJ7Cy4ohQ866YVa2UQUDAtN5EPvzmZJ0VZNCqp7WfNFxbWf94YfHESCFjPv8ux1z8pgv1PCTppkuodbj3adj0TBH57ouEnM9Qag3cxhAJAZw1CqTtTgYbC9EAme15Z6awbcDVBdJh6tksTwe4Xpjdw7fyfVYD55aPhCd1stuV0R1pCATBPTofbpK86GfN4mQA29HghH6YjCpKxn16XenXFY2BF4hdWmSLvzavusfeB6BUj1bv7jsfKtTp2j06712EURgfCttrQfG46cfrQw4fxnaga33aj9nPTRoiyn1ED6q7QX0h9s4to7351jsULdj8BUK135655ga48Hu9KfW5E9g5rWbs9rv3EufE81XuwpdR89p2Gj1W1oNAL6P1wJgOq3FHE4X12APcwoXTPvzT6xP5x9EMsVbVWZoFNaZNaTjy514W22yRoYhNnAYRTJ2t15m5vpxLJUttcm84vhGka1G3PRDyYh14ZMwK6G0e2fr2ubshpZpHf8hq42znov2U9uxvPP5cxqe5r71dm8DPAG8XForC0jx2vPKmPMW5v1ne5c1rFY9bAbJwr89xVZ48BkXmRmgncpX1WKWxwneSefU1.16LRYG335043jrc3y3SznKpMnyC8H1tHJLTAJG45DxpU18nav2DjhxcCl2jXelUjBqmHsz3K7oZ2gTAvpXtbdg5kT41nETKvruqUZakHJYbCSWZK4KYkBoRrKvNkl46DxrXNkBDvqthCv8MHfe1Hq3tsylK63ABpVbV4dCNX9DBRAPpQ6KsZ3C415fgmopD0X2TSJNJpa8YUG3k7or2gv8Pq6df5qLTSBxkdFQ4h2ug0Zdsolw4dVxTB34rZX2VhLkBLYP94Z8M4t3oFgtcU02Rpprx36Gxa11c1105X1P1boHMK5XBGA1uy57dQzajBUKf1nsL7KM6o7vcve22uXukxsslygWmvrYogp8gozRSbsCFXbZf5mZMSKjGdsTcsJmK5Ab7hnc0XEFZcbo0z3zvgYnLq0sLH1btr5RK995Gwx9UDUfytDaMzv8DeoYxz6drSbsmCZmLmZ8jNyzr74hvZABqKFWDKFF1LXAhQvnt1qK6j1hVgGGAJxd2bZdLU96W7W2cRARRxBxbtTY2barkSwwF25UttPbzqzxdQMXD5srQlwxH3KGHz6y45Yz

Receive address

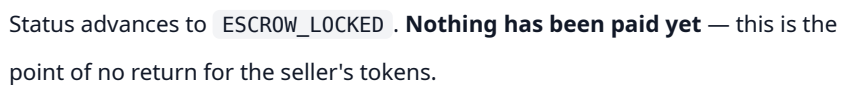
BcNyDemoBuyerReceiveAddress

After **Confirm buy** the engine mints the swap at **MATCHED** and derives the 2-of-3 escrow address from the seller's and buyer's PQ dids plus the engine's prefixed ML-DSA public key (**SETTLEMENT_ENGINE_PUBKEY**) — shown truncated on the card:

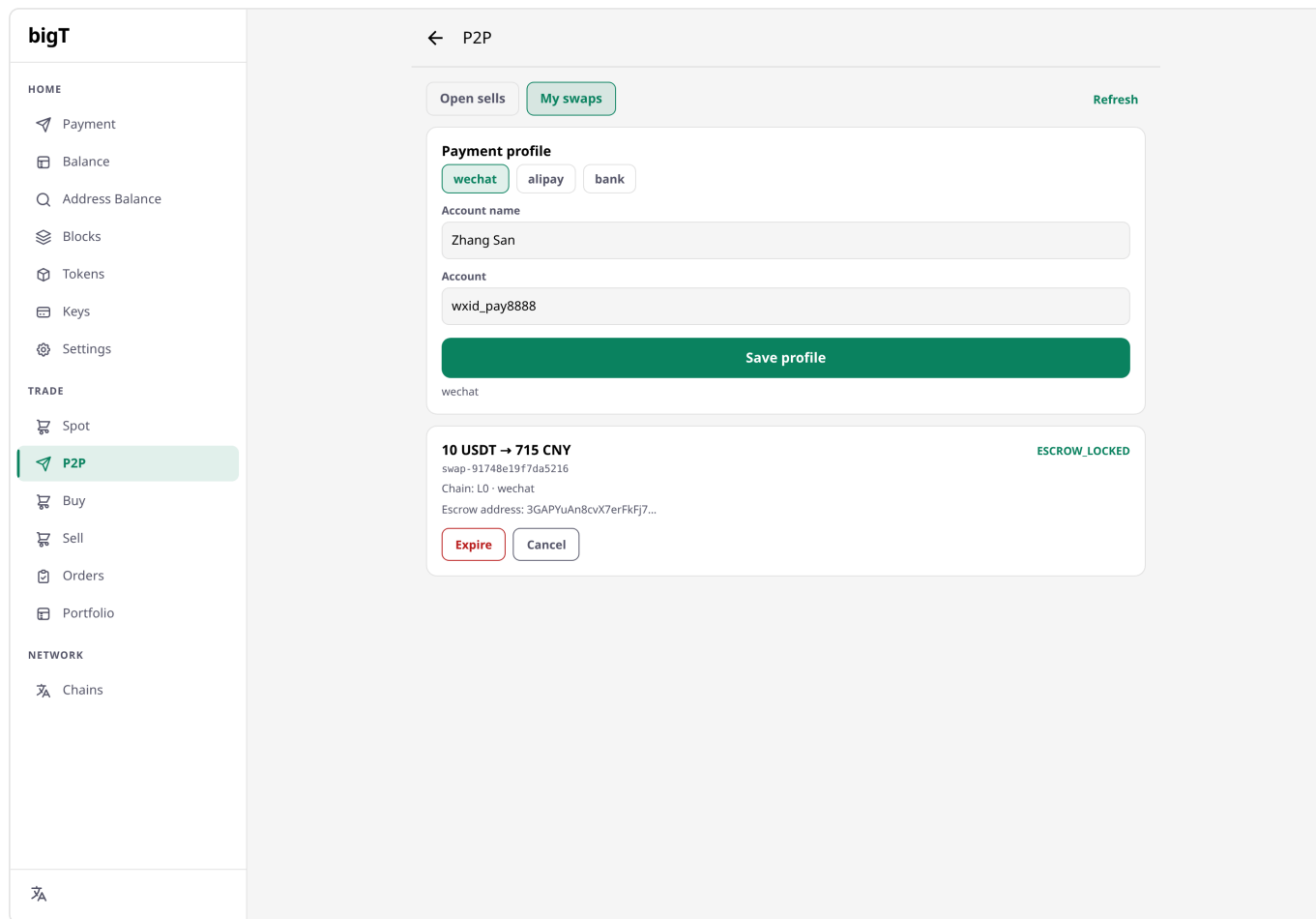


Step 4 — Seller locks the token escrow

Back on the seller's **My swaps** tab, the seller pastes the escrow funding transaction hash and presses **Lock escrow** — a seller-signed `action=escrow_lock` transition. With `deps.chain` configured the engine verifies the lock on chain; in the demo (`SETTLEMENT_STORE=mem` , no chain) it is recorded as-is.



Status advances to `ESCROW_LOCKED` . **Nothing has been paid yet** — this is the point of no return for the seller's tokens.



Step 5 — Buyer pulls the signed payment instructions

The buyer's card now shows a **Payment details** button. Pressing it issues the buyer-signed `POST /swaps/:id/payment-instructions`, which is idempotent while `PAYMENT_PENDING` and reveals the seller's profile + a fresh per-swap `remark` the transfer *must* carry:

```
{ "method": "wechat", "rail": "wechat", "accountName": "Zhang San",  
  "account": "wxid_pay8888", "amount": "715", "currency": "CNY",  
  "remark": "0a42eb4b426c", "payBy": 1791510060 }
```

bigT

HOME

Payment

Balance

Address Balance

Blocks

Tokens

Keys

Settings

TRADE

Spot

P2P

Buy

Sell

Orders

Portfolio

NETWORK

Chains

←

P2P

Open sells

My swaps

Refresh

Payment profile

wechat

alipay

bank

Account name

Account

Save profile

10 USDT → 715 CNY

swap-91748e19f7da5216

Chain: L0 · wechat

Escrow address: 3GAPYuAn8cvX7erFkJ7...

ESCROW_LOCKED

Payment details

Cancel

The instructions render **inline in the swap card** (amount, account, remark, deadline) — plus an optional QR payload when the profile stores one:

bigT

HOME

Payment

Balance

Address Balance

Blocks

Tokens

Keys

Settings

TRADE

Spot

P2P

Buy

Sell

Orders

Portfolio

NETWORK

Chains

← P2P

Open sells

My swaps

Refresh

Payment profile

wechat

alipay

bank

Account name

Account

Save profile

10 USDT → 715 CNY

PAYMENT_PENDING

swap-91748e19f7da5216

Chain: L0 · wechat

Escrow address: 3GAPYuAn8cvX7erFkJ7...

Remark: 910fc8a8926f

Pay the seller

Account name: Zhang San

Account: wxid_pay8888

715 CNY

Remark: 910fc8a8926f

Pay by: 10/9/2026, 3:54:48 PM

Transaction ID

I have paid

Dispute

Step 6 — Buyer pays in WeChat and claims with the 流水号

The buyer transfers 715 CNY in their own WeChat app, pastes the transaction number (流水号) into **Transaction ID**, and presses **I have paid** (POST /swaps/:id/proof). The optional receipt image is stored as-is in the proof store; only its sha256 is anchored on the swap event.

bigT

HOME

Payment

Balance

Address Balance

Blocks

Tokens

Keys

Settings

TRADE

Spot

P2P

Buy

Sell

Orders

Portfolio

NETWORK

Chains

←

P2P

Open sells

My swaps

Refresh

Payment profile

wechat

alipay

bank

Account name

Account

Save profile

10 USDT → 715 CNY

PAYMENT_PENDING

swap-91748e19f7da5216

Chain: L0 · wechat

Escrow address: 3GAPYuAn8cvX7erFkJ7...

Remark: 910fc8a8926f

Pay the seller

Account name: Zhang San

Account: wxid_pay8888

715 CNY

Remark: 910fc8a8926f

Pay by: 10/9/2026, 3:54:48 PM

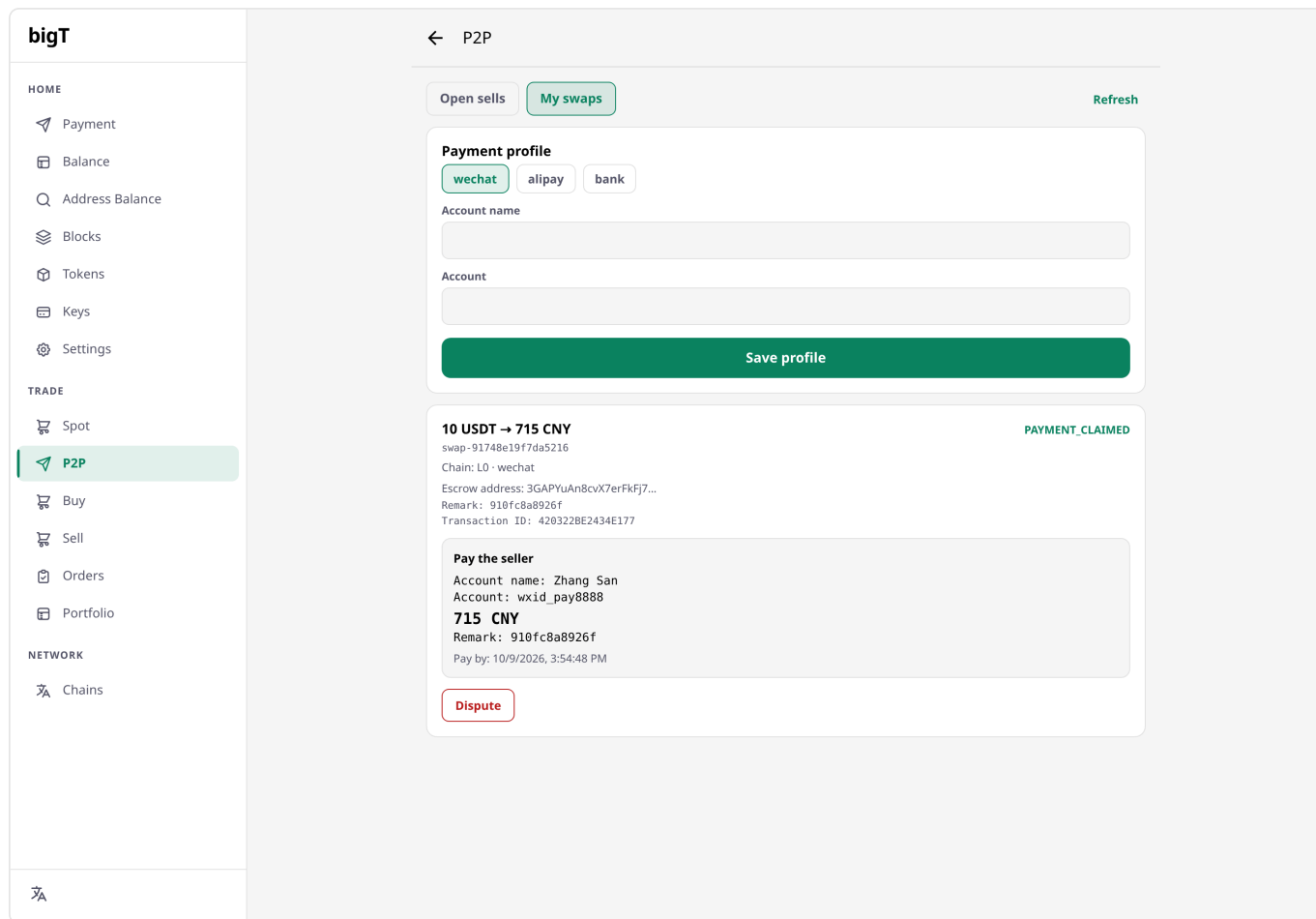
Transaction ID

420322BE2434E177

I have paid

Dispute

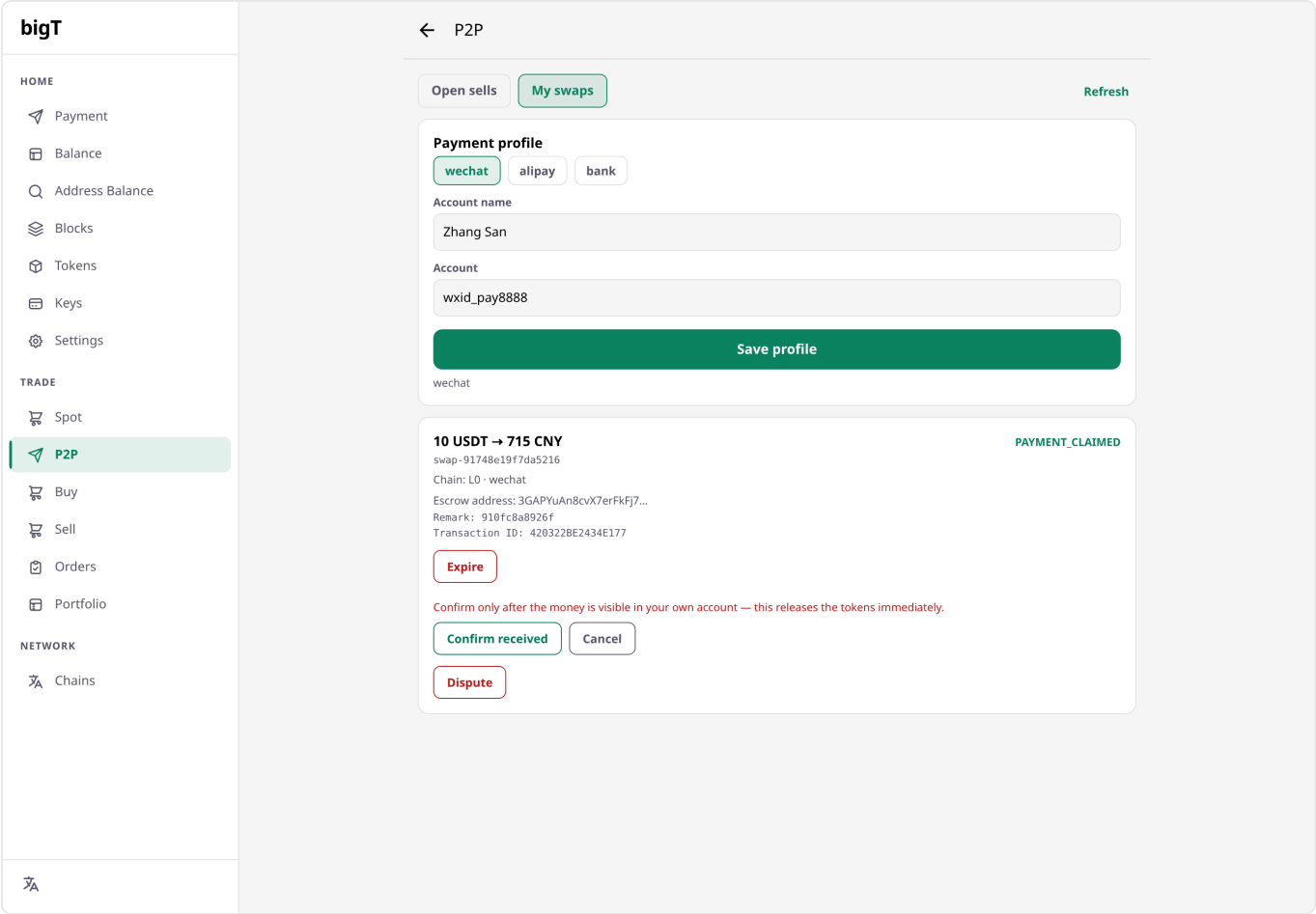
Status is now `PAYMENT_CLAIMED` — the tokens are still locked. This state exists purely as the buyer's claim.



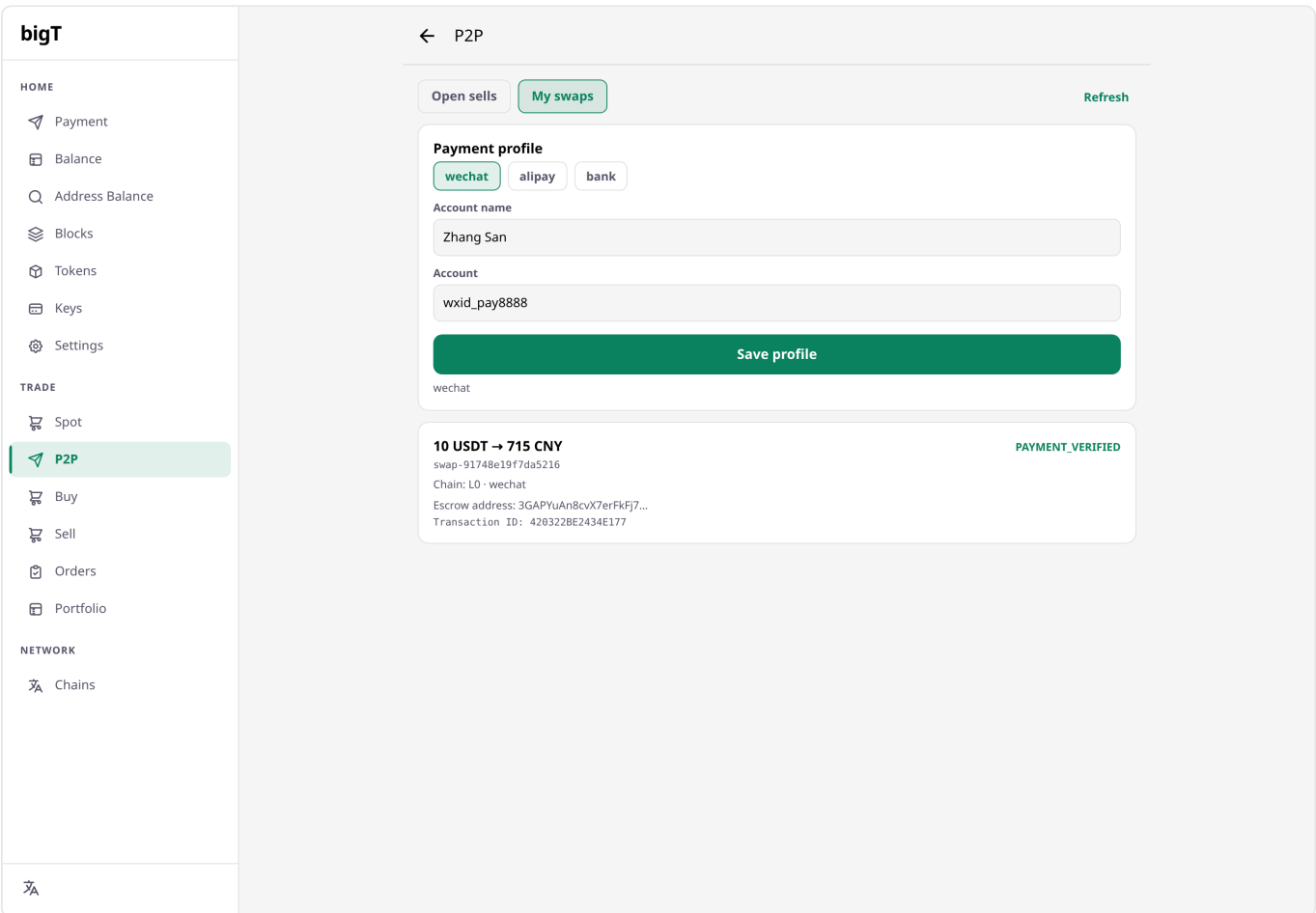
Step 7 — Seller verifies on their own statement, then confirms

The seller's card shows the buyer's 流水号 and the remark. The seller checks **amount + remark + time + 流水号 in their own WeChat account** (screenshots never count), and only then presses **Confirm received** — a seller-signed `POST /swaps/:id/confirm`. The red warning above the button is the whole safety story of the CNY rail:

Confirm only after the money is visible in your own account — this releases the tokens immediately.



PAYMENT_VERIFIED :



Step 8 — Engine releases the tokens, swap completes

release and complete are engine-signed actions — the wallet deliberately holds no key for them (see §6.1: nothing in the engine fires them automatically yet). The capture script therefore stands in for the operator / release job and signs both with the SETTLEMENT_ENGINE_DID key:

```
POST /swaps/swap-62d84d9fb89e9320/transitions # did = engine, action=release, txHash=...
→ { "status": "ESCROW_RELEASED" }
POST /swaps/swap-62d84d9fb89e9320/transitions # did = engine, action=complete
→ { "status": "COMPLETED" }
```

bigT

HOME

Payment

Balance

Address Balance

Blocks

Tokens

Keys

Settings

TRADE

Spot

P2P

Buy

Sell

Orders

Portfolio

NETWORK

Chains

← P2P

Open sells

My swaps

Refresh

Payment profile

wechat

alipay

bank

Account name

Account

Save profile

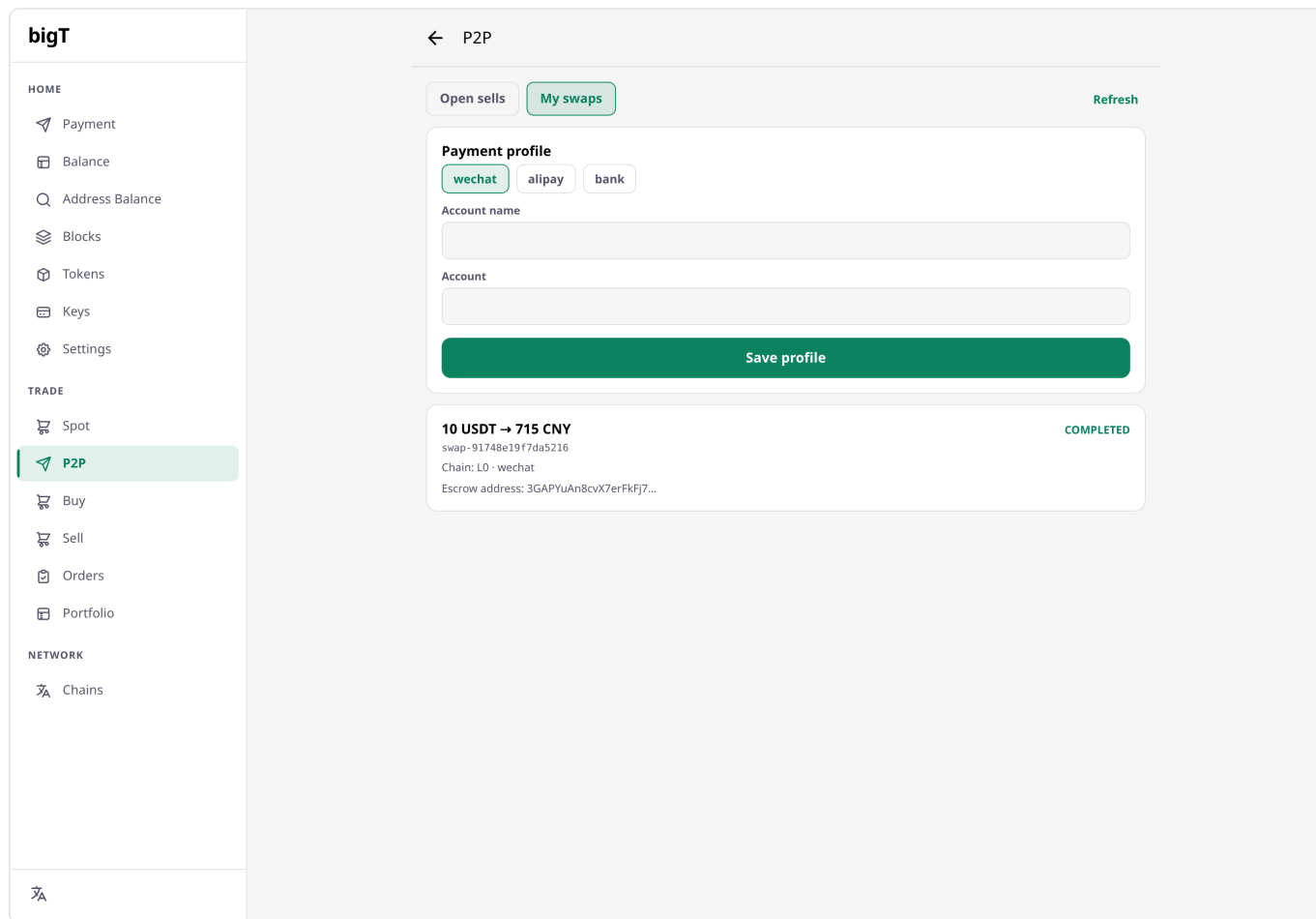
10 USDT → 715 CNY

ESCROW_RELEASED

swap-91748e19f7da5216

Chain: L0 · wechat

Escrow address: 3GAPYUan8cvX7erFkJ7...



Timeline of the captured run

Time	Step
07:38:23	both wallets created
07:38:27	seller payment profile saved (wechat)
07:38:31	sell order listed (wantRail=wechat , 715 CNY)
07:38:34	buyer matched the order (no paypalAccount / buyerEmail)
07:39:47	escrow lock recorded (ESCROW_LOCKED)
07:39:49	buyer pulled payment instructions (户名 / account / 715 CNY / remark)
07:39:51	buyer claimed payment (流水号 420322BE2434E177)
07:39:53	seller confirmed receipt (PAYMENT_VERIFIED)
07:39:53	engine signed the release (ESCROW_RELEASED)
07:39:55	swap completed (COMPLETED) — seller already holds the CNY

(The 70 s gap after matching is deliberate: the engine allows 10 signed requests per DID per 60 s and the seller is at that ceiling after profile + list + match.)

5. HTTP API reference (CNY rails)

All mutating endpoints take a signed body:

```
{ /* endpoint-specific fields, e.g. action / txId / ... */
  "did": "did:key:z446G...",           // ML-DSA-87 did:key – the signer (see table §1)
  "nonce": "b1f0...",                 // ≥16 chars, single use
  "timestamp": 1791509100000,          // ms, ±5 min (sign.ts NONCE_MAX_SKEW_MS)
  "signature": "...hex SignatureBundle..." }
```

`signature` = `PQ: SHA256(JSON.stringify(payload, sortedKeys))` signed with the did's ML-DSA-87 key, hex-serialized `SignatureBundle` (`sign.ts:39-61` , mirrors the wallet's `p2pIdentity.signP2pPayload`). A classic did instead verifies an Ed25519 signature over the raw bytes, or an ECDSA signature over their sha256 — supported for compatibility, not what new keys use. `payload` is always the body **without** `signature` .

Endpoint	Signer	Required fields	Success
GET /healthz	—	—	{ ok: true }
GET /public/orders	—	—	{ orders: [...] } (no signature, no PII)
POST /profiles	seller	method , accountName , account (+ bankName , qr)	{ ok, method, updatedAt }
POST /profiles/mine	seller	—	{ profiles: [...] }
POST /orders	seller	giveToken , giveAmount , giveChain , wantCurrency , wantAmount , wantRail , validUntil	201 { orderId, status }
POST /orders/:id/match	buyer	receiveAddress (CNY: paypalAccount / buyerEmail not used)	201 { swapId, status, escrowAddress }
POST /swaps/mine	party	—	{ swaps: [...] }
POST /swaps/:id/transitions	seller (lock/expire/refund) · either (cancel) · engine (release/complete/verify)	action , + txHash for lock/release	{ swapId, status }
POST /swaps/:id/payment-instructions	buyer	—	201 { ... instructions, remark, payBy }
POST /swaps/:id/proof	buyer	txId (1..64 [A-Za-z0-9._-]), optional remark , receipt , receiptSha256	{ swapId, status, txId, receiptSha256 }
POST /swaps/:id/confirm	seller	—	{ swapId, status: "PAYMENT_VERIFIED" }
POST /swaps/:id/dispute	either party	optional reason (≤512)	{ swapId, dispute }
POST /swaps/:id/dispute/resolve	admin token	`outcome=release`	refund`
			{ swapId, ... }

Guard rails the engine enforces on every one of them:

- 409 when the status transition is illegal (`state.ts` `VALID_TRANSITIONS`),

409 on a dedicated action sent to the generic route (`instructions` ,
`payment_proof` , `payment_confirm` are rejected there),

- 403 unless `did` is the required signer, 400 on bad signature, replayed
nonce, stale timestamp, or `rate limit exceeded` (10/min/DID),
- 422 swap frozen: ... while a dispute is open for the forward actions,
- 400 for rail/action mismatches (`CNY swap: use .../proof` ,
`complete` is for CNY swaps).

6. Review findings (code review of the CNY implementation)

Verified good: the transition table keeps `PAYMENT_CLAIMED` out of the release
path; `release` / `complete` are engine-only; the profile is party-scoped; the
remark is regenerated per swap and checked on `proof` ; the receipt hash is
recomputed server-side; unit tests cover the CNY legs
(`services/p2p-engine/test/cny.test.ts` , 54 engine tests green).

Open gaps, in the order they matter:

1. **No automatic release after `confirm`**. The engine does not issue
`release` / `complete` itself when the seller confirms — an operator/cron must
do it with the engine key. Today the wallet UI has no button for it either
(by design: the wallet must not hold the engine key).
2. **`docs/p2pcny.md` was stale** — it still said "Status: plan — not built",
documented the instructions route as `GET` , and allowed disputes from "any
pre-release state". Corrected in this pass (status, `POST` route, dispute
states `PAYMENT_PENDING` / `PAYMENT_CLAIMED` , `confirm` no longer claiming to
broadcast the release); `SETTLEMENT_CNY_CONFIRM_TIMEOUT` and
`SETTLEMENT_CNY_MAX_CENTS` are now marked *not implemented* in that doc.
3. **`payBy` is advisory**. The `SETTLEMENT_CNY_REMARK_TTL` deadline is returned
to the buyer but never enforced — a swap can be claimed after it expires,
and `SETTLEMENT_CNY_MAX_CENTS` / a confirm timeout are not implemented.
4. **`receiptSha256` hashed the data-URL string, not the image bytes** — fixed
in this pass: `server.ts` decodes the `data:image/...;base64,...` payload and
hashes the decoded bytes (400 on malformed base64), so the digest is
reproducible with `sha256 <file>` ; `test/cny.test.ts` asserts it.
5. **`Alert.alert` is a no-op on react-native-web** — every error/success
message in the P2P screen (including the payment instructions) was invisible
in the web build. Fixed in this pass: inline notice banner + inline
instructions panel + confirm warning, localized in 12 languages
(`p2p.payBy` , `p2p.confirmWarn`).
6. **Rate limit bites scripted flows**. 10 signed calls/DID/minute is tight for
the wallet's own read-modify loops (`/swaps/mine` after every transition);
the capture script has to pace itself. Worth making `RATE_LIMIT` an env knob

if real users ever hit it.

7. Regenerating this guide

```
node e2e/capture-p2p-cny.mjs          # refresh the 14 screenshots
node docs/p2p-demo/scripts/gen-p2p-cny-pdf.mjs
# → docs/p2p-demo/assets/p2p-cny.pdf
```

The generator is dependency-free (own markdown → HTML, images embedded as data

URIs, `chromium --headless --print-to-pdf`), the same pipeline the dai help

guides use.